

LLM-Driven TLA+ Specification of Essential Paxos

Ana Mako

Vuk Jurišić

Giorgos Markozanis

AGENDA

- Research Question
- Problem Statement
- SysMoBench's four-stage pipeline
- Experiment 1
- Experiment 2
- Experiment 3
- Limitations
- Conclusion

RESEARCH QUESTION

Does feedback-driven prompt refinement improve the quality of LLM-generated TLA+ specifications, and are the resulting prompts transferable across Claude and GPT?

PROBLEM STATEMENT

- Safety properties must hold for every execution, testing isn't enough

PROBLEM STATEMENT

- Safety properties must hold for every execution, testing isn't enough
- TLA+ and TLC model checking gives exhaustive guarantees, but writing the spec needs dual expertise

PROBLEM STATEMENT

- Safety properties must hold for every execution, testing isn't enough
- TLA+ and TLC model checking gives exhaustive guarantees, but writing the spec needs dual expertise
- **Idea:** have an LLM generate the spec from the implementation

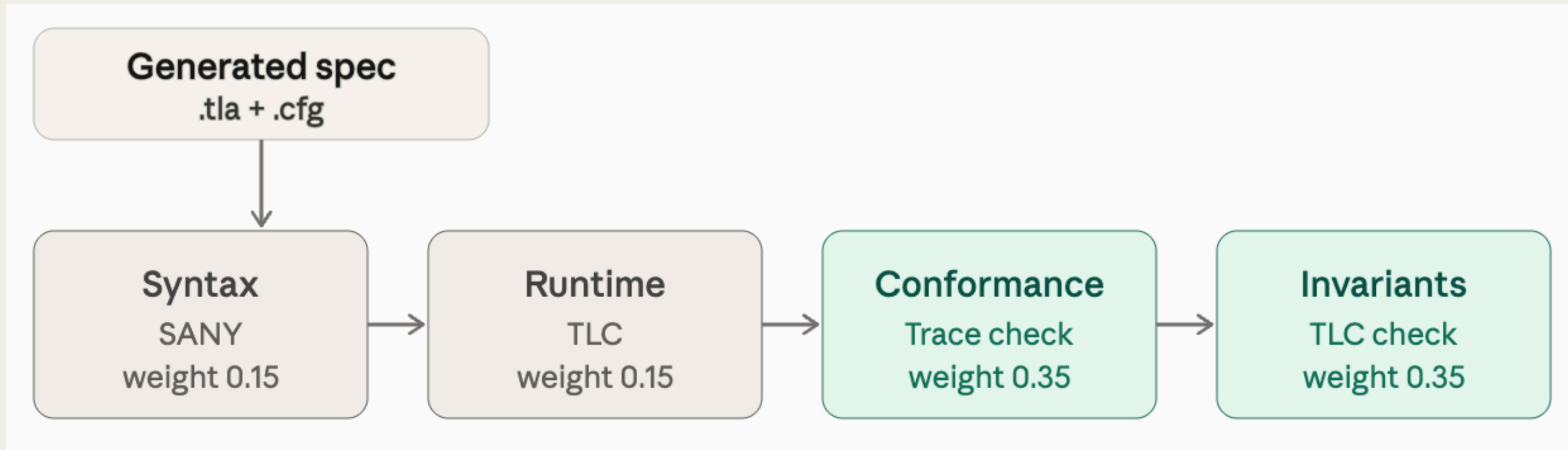
PROBLEM STATEMENT

- Safety properties must hold for every execution, testing isn't enough
- TLA+ and TLC model checking gives exhaustive guarantees, but writing the spec needs dual expertise
- **Idea:** have an LLM generate the spec from the implementation
- **Case study:** Essential Paxos (202-line core of cocagne/paxos)

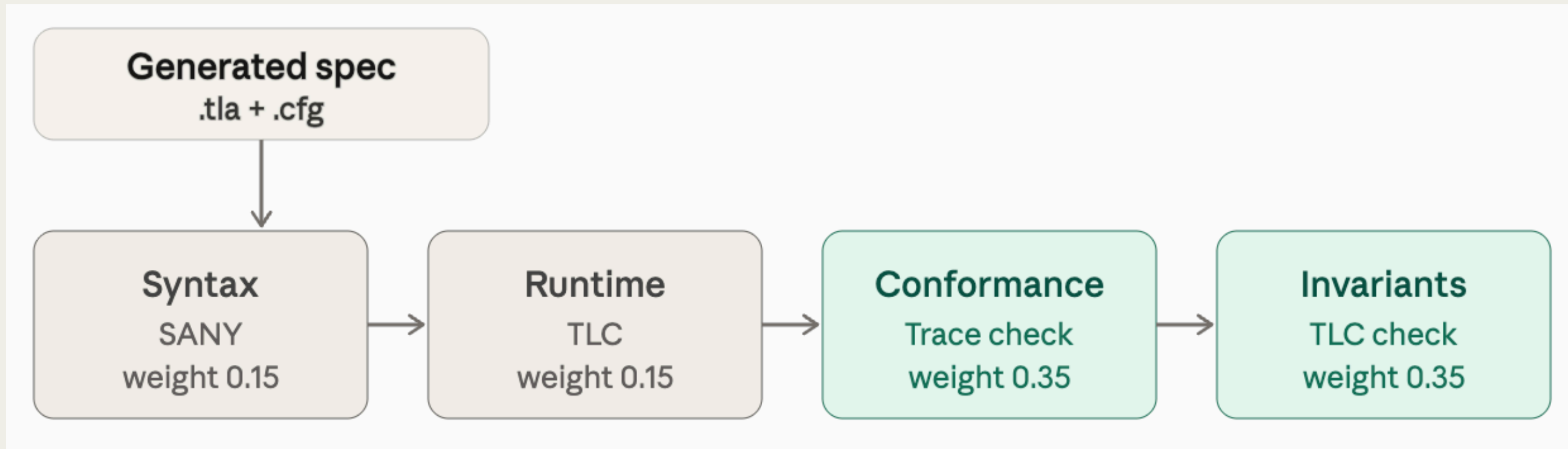
PROBLEM STATEMENT

- Safety properties must hold for every execution, testing isn't enough
- TLA+ and TLC model checking gives exhaustive guarantees, but writing the spec needs dual expertise
- **Idea:** have an LLM generate the spec from the implementation
- **Case study:** Essential Paxos (202-line core of cocagne/paxos)
- **We compare two strategies:** feedback-driven refinement vs. single-pass prompt

SYSMOBENCH'S FOUR-STAGE PIPELINE

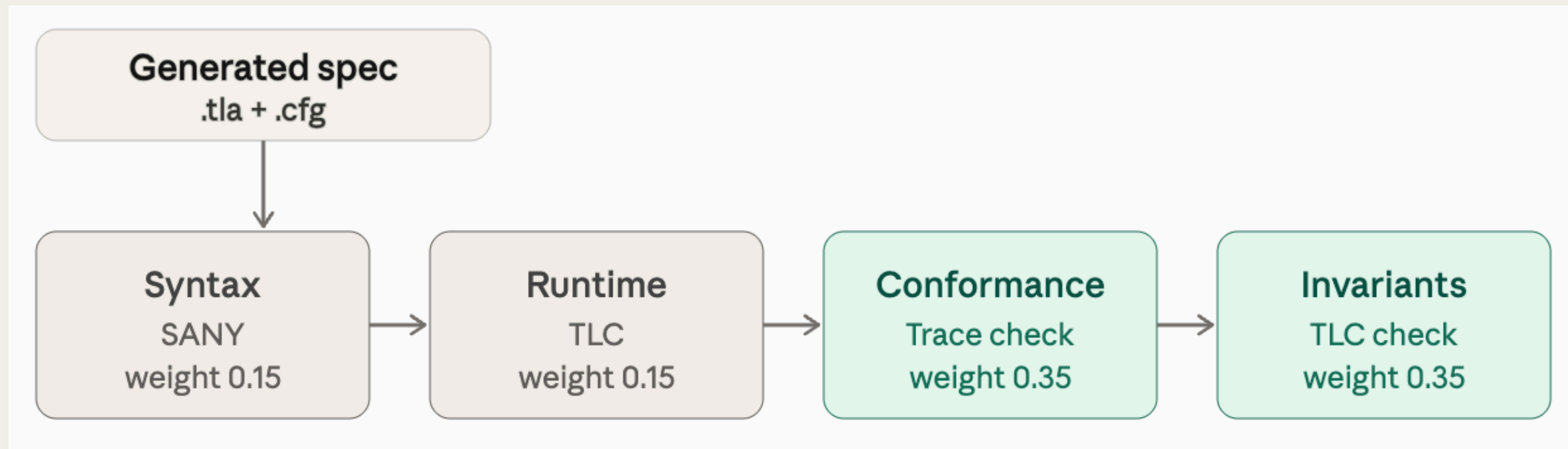


SYSMOBENCH'S FOUR-STAGE PIPELINE



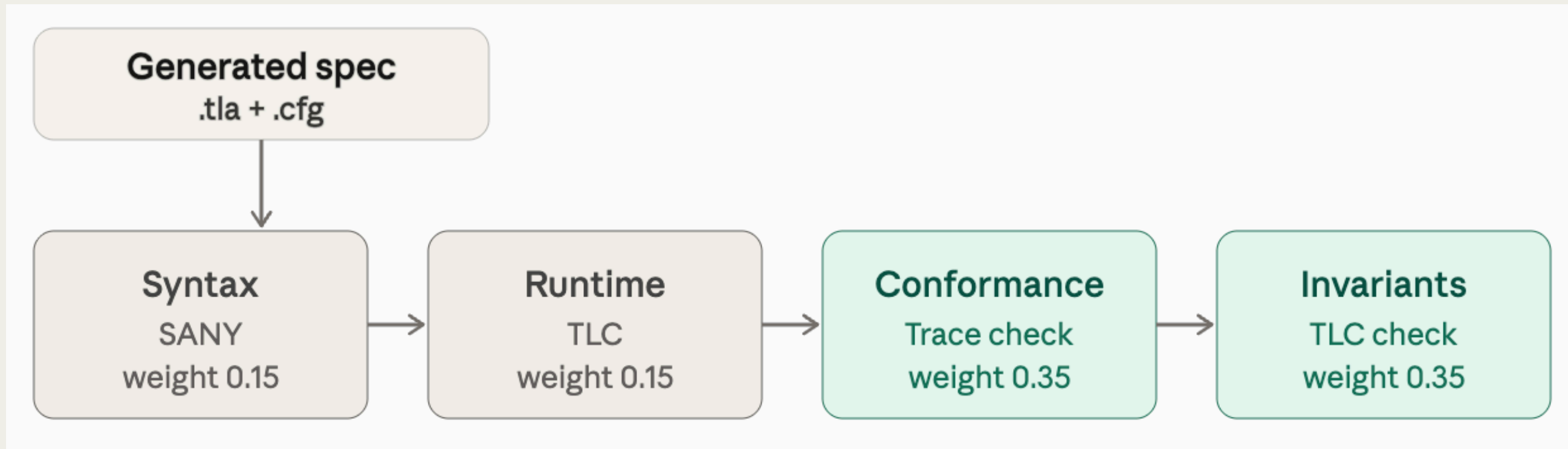
- Syntax + runtime are necessary but not sufficient for a faithful spec

SYSMOBENCH'S FOUR-STAGE PIPELINE



- Syntax + runtime are necessary but not sufficient for a faithful spec
- The two heavy stages test conformance to real execution traces and expert safety invariants

SYSMOBENCH'S FOUR-STAGE PIPELINE



- Syntax + runtime are necessary but not sufficient for a faithful spec
- The two heavy stages test conformance to real execution traces and expert safety invariants
- TV uses actual Python execution traces as ground truth

EXPERIMENT 1: INITIAL RUN

EXPERIMENT 1: INITIAL RUN

CLAUDE

Run	Outcome	Root cause
1	FAIL	Learners not assigned in .cfg
2	FAIL	Unbounded CHOOSE $n \in \text{Nat}$
3	FAIL	$>$ applied to model value
4	FAIL	Infinite state space
5	FAIL	SANY parse error
6	FAIL	SANY parse error
7	FAIL	TLC exception
8	PASS	All four metrics passing

EXPERIMENT 1: INITIAL RUN

CLAUDE

Run	Outcome	Root cause
1	FAIL	Learners not assigned in .cfg
2	FAIL	Unbounded CHOOSE $n \in \text{Nat}$
3	FAIL	$>$ applied to model value
4	FAIL	Infinite state space
5	FAIL	SANY parse error
6	FAIL	SANY parse error
7	FAIL	TLC exception
8	PASS	All four metrics passing

GPT

Run	Outcome	Root cause
1	FAIL	TLA+ did not compile
2	PARTIAL	TV/invariants failed
3	PARTIAL	Malformed invariants
4	PASS	All four metrics passing

EXPERIMENT 1: INITIAL RUN

CLAUDE

Run	Outcome	Root cause
1	FAIL	Learners not assigned in .cfg
2	FAIL	Unbounded CHOOSE $n \in \text{Nat}$
3	FAIL	$>$ applied to model value
4	FAIL	Infinite state space
5	FAIL	SANY parse error
6	FAIL	SANY parse error
7	FAIL	TLC exception
8	PASS	All four metrics passing

GPT

Run	Outcome	Root cause
1	FAIL	TLA+ did not compile
2	PARTIAL	TV/invariants failed
3	PARTIAL	Malformed invariants
4	PASS	All four metrics passing

- Neither model passed on the first prompt (8 iterations (Claude), 4 iterations (GPT))

EXPERIMENT 1: INITIAL RUN

CLAUDE

Run	Outcome	Root cause
1	FAIL	Learners not assigned in .cfg
2	FAIL	Unbounded CHOOSE $n \in \text{Nat}$
3	FAIL	$>$ applied to model value
4	FAIL	Infinite state space
5	FAIL	SANY parse error
6	FAIL	SANY parse error
7	FAIL	TLC exception
8	PASS	All four metrics passing

GPT

Run	Outcome	Root cause
1	FAIL	TLA+ did not compile
2	PARTIAL	TV/invariants failed
3	PARTIAL	Malformed invariants
4	PASS	All four metrics passing

- Neither model passed on the first prompt (8 iterations (Claude), 4 iterations (GPT))
- Shared failure class: finite-domain encoding (bounded ballots, no model-value comparison, cardinality-based quorums)

EXPERIMENT 1: INITIAL RUN

CLAUDE

Run	Outcome	Root cause
1	FAIL	Learners not assigned in .cfg
2	FAIL	Unbounded CHOOSE $n \in \text{Nat}$
3	FAIL	$>$ applied to model value
4	FAIL	Infinite state space
5	FAIL	SANY parse error
6	FAIL	SANY parse error
7	FAIL	TLC exception
8	PASS	All four metrics passing

GPT

Run	Outcome	Root cause
1	FAIL	TLA+ did not compile
2	PARTIAL	TV/invariants failed
3	PARTIAL	Malformed invariants
4	PASS	All four metrics passing

- Neither model passed on the first prompt (8 iterations (Claude), 4 iterations (GPT))
- Shared failure class: finite-domain encoding (bounded ballots, no model-value comparison, cardinality-based quorums)
- Feedback loop is autonomous: each SANY/TLC failure is fed back, the model regenerates

EXPERIMENT 2: CROSS-MODEL PROMPT TRANSFER

EXPERIMENT 2: CROSS-MODEL PROMPT TRANSFER

Model	Prompt	Compile	Runtime	TV rate	Invariants
Claude Sonnet 4.5	Prompt _C	PASS	PASS	78.7% (48/61)	4/4
Claude Sonnet 4.5	Prompt _X	FAIL	—	—	—
GPT-5	Prompt _C	FAIL	—	—	—
GPT-5	Prompt _X	PASS	PASS	100% (61/61)	4/4

EXPERIMENT 2: CROSS-MODEL PROMPT TRANSFER

Model	Prompt	Compile	Runtime	TV rate	Invariants
Claude Sonnet 4.5	Prompt _C	PASS	PASS	78.7% (48/61)	4/4
Claude Sonnet 4.5	Prompt _X	FAIL	—	—	—
GPT-5	Prompt _C	FAIL	—	—	—
GPT-5	Prompt _X	PASS	PASS	100% (61/61)	4/4

- Each prompt compiles only on the model that produced it
- Both cross-model combinations fail at compilation
- Prompts encode model-specific generation habits, not portable spec knowledge

EXPERIMENT 2: CROSS-MODEL PROMPT TRANSFER

Model	Prompt	Compile	Runtime	TV rate	Invariants
Claude Sonnet 4.5	Prompt _C	PASS	PASS	78.7% (48/61)	4/4
Claude Sonnet 4.5	Prompt _X	FAIL	—	—	—
GPT-5	Prompt _C	FAIL	—	—	—
GPT-5	Prompt _X	PASS	PASS	100% (61/61)	4/4

- Each prompt compiles only on the model that produced it
- Both cross-model combinations fail at compilation
- Prompts encode model-specific generation habits, not portable spec knowledge

Winner: GPT-5 + PromptX - the only fully faithful spec (100%TV, 4/4 Invariants)

EXPERIMENT 2: WHY CROSS-MODEL FAILS

EXPERIMENT 2: WHY CROSS-MODEL FAILS

Claude + PromptX → compile FAIL

Helper uses proposalId before VARIABLES declares it

```
NextProposalNum(p) ==  
  IF proposalId[p] = NoProposal  
  THEN 1  
  ELSE ProposalNum(proposalId[p]) + 1
```

```
VARIABLES  
  msgs, proposalId, ...
```

SANY: Unknown operator 'proposalId'

EXPERIMENT 2: WHY CROSS-MODEL FAILS

Claude + PromptX → compile FAIL

Helper uses proposalId before VARIABLES declares it

```
NextProposalNum(p) ==  
  IF proposalId[p] = NoProposal  
  THEN 1  
  ELSE ProposalNum(proposalId[p]) + 1  
  
VARIABLES  
  msgs, proposalId, ...
```

SANY: Unknown operator 'proposalId'

GPT-5 + PromptC → compile FAIL

The two prompts use different no-proposal sentinels

```
\* Prompt_X (GPT): in-module tuple  
NoProposal == <<0, 0>>  
\* do NOT bind NoProposal in the cfg  
  
\* Prompt_C (Claude): model value  
CONSTANTS ... NoValue \* bound in cfg
```

Crossing the conventions → invalid TLA+

EXPERIMENT 2: WHY CROSS-MODEL FAILS

Claude + PromptX → compile FAIL

Helper uses proposalId before VARIABLES declares it

```
NextProposalNum(p) ==  
  IF proposalId[p] = NoProposal  
  THEN 1  
  ELSE ProposalNum(proposalId[p]) + 1  
  
VARIABLES  
  msgs, proposalId, ...
```

SANY: Unknown operator 'proposalId'

GPT-5 + PromptC → compile FAIL

The two prompts use different no-proposal sentinels

```
\* Prompt_X (GPT): in-module tuple  
NoProposal == <<0, 0>>  
\* do NOT bind NoProposal in the cfg  
  
\* Prompt_C (Claude): model value  
CONSTANTS ... NoValue \* bound in cfg
```

Crossing the conventions → invalid TLA+

Failures are fingerprints, not noise. Each prompt bakes in its own model's generation order and sentinel style.

EXPERIMENT 3: CONFIGURATION SCALING

EXPERIMENT 3: CONFIGURATION SCALING

Configuration	Small	Baseline	Large
Acceptors	3	3	5
Proposers	1	2	3
Learners	1	1	1
MaxBallot	2	3	3
Values	2	2	2

Table 5: TLC model checking results under 90 s cap (both specs)

Config	Claude + Prompt _C			GPT-5 + Prompt _X		
	States	Distinct	Done?	States	Distinct	Done?
Small	1,217	244	Yes (<1 s)	1,857,794	72,764	Yes (~36 s)
Baseline	1,502,755	222,984	Yes (~26 s)	>3,680,568	>397,309	No
Large	>3,696,573	>779,841	No	>2,548,004	>441,963	No

- Identical configs, very different cost: ~1.86M states for GPT at Small vs 1217 for Claude
- Within 90s: Claude finishes Small + Baseline; GPT finishes only Small

EXPERIMENT 3: CONFIGURATION SCALING

Configuration	Small	Baseline	Large
Acceptors	3	3	5
Proposers	1	2	3
Learners	1	1	1
MaxBallot	2	3	3
Values	2	2	2

Table 5: TLC model checking results under 90 s cap (both specs)

Config	Claude + Prompt _C			GPT-5 + Prompt _X		
	States	Distinct	Done?	States	Distinct	Done?
Small	1,217	244	Yes (<1 s)	1,857,794	72,764	Yes (~36 s)
Baseline	1,502,755	222,984	Yes (~26 s)	>3,680,568	>397,309	No
Large	>3,696,573	>779,841	No	>2,548,004	>441,963	No

- Identical configs, very different cost: ~1.86M states for GPT at Small vs 1217 for Claude
- Within 90s: Claude finishes Small + Baseline; GPT finishes only Small
- **Cause:** GPT's HandlePromise stutter branch (message loss) doubles branching on every Promise

EXPERIMENT 3: CONFIGURATION SCALING

Configuration	Small	Baseline	Large
Acceptors	3	3	5
Proposers	1	2	3
Learners	1	1	1
MaxBallot	2	3	3
Values	2	2	2

Table 5: TLC model checking results under 90 s cap (both specs)

Config	Claude + Prompt _C			GPT-5 + Prompt _X		
	States	Distinct	Done?	States	Distinct	Done?
Small	1,217	244	Yes (<1 s)	1,857,794	72,764	Yes (~36 s)
Baseline	1,502,755	222,984	Yes (~26 s)	>3,680,568	>397,309	No
Large	>3,696,573	>779,841	No	>2,548,004	>441,963	No

- Identical configs, very different cost: ~1.86M states for GPT at Small vs 1217 for Claude
- Within 90s: Claude finishes Small + Baseline; GPT finishes only Small
- **Cause:** GPT's HandlePromise stutter branch (message loss) doubles branching on every Promise
- **Tradeoff:** GPT's 100% TV spec is far costlier to verify than Claude's 78.7%

EXPERIMENT 3: WHY IT EXPLODES

EXPERIMENT 3: WHY IT EXPLODES

GPT's HandlePromise: two disjuncts

```
HandlePromise(p, m) ==  
  /\ m \in msgs  
  /\ m.kind = "Promise"  
  /\ p \in Proposers  
  /\ m.to = p  
  /\ ( \* process a fresh promise  
      /\ m.pid = proposalId[p]  
      /\ ~(m.from \in promisesRcvd[p])  
      /\ LET ... IN \* update state;  
          \* emit Accept msgs  
  /\ \* ignore duplicates or old promises  
  /\ UNCHANGED vars  
  )
```


EXPERIMENT 3: WHY IT EXPLODES

GPT's HandlePromise: two disjuncts

```
HandlePromise(p, m) ==  
  /\ m \in msgs  
  /\ m.kind = "Promise"  
  /\ p \in Proposers  
  /\ m.to = p  
  /\ ( \* process a fresh promise  
      /\ m.pid = proposalId[p]  
      /\ ~(m.from \in promisesRcvd[p])  
      /\ LET ... IN \* update state;  
          \* emit Accept msgs  
  /\ \* ignore duplicates or old promises  
  /\ UNCHANGED vars  
  )
```

- Disjunct (2) has no guard — every Promise can be processed OR ignored
- TLC must explore both → branching doubles on every message

EXPERIMENT 3: WHY IT EXPLODES

GPT's HandlePromise: two disjuncts

```
HandlePromise(p, m) ==  
  /\ m \in msgs  
  /\ m.kind = "Promise"  
  /\ p \in Proposers  
  /\ m.to = p  
  /\ ( \* process a fresh promise  
      /\ m.pid = proposalId[p]  
      /\ ~(m.from \in promisesRcvd[p])  
      /\ LET ... IN \* update state;  
          \* emit Accept msgs  
  /\ \* ignore duplicates or old promises  
  /\ UNCHANGED vars  
  )
```

- Disjunct (2) has no guard — every Promise can be processed OR ignored
- TLC must explore both → branching doubles on every message
- Same config, Small: 1,217 states (Claude) vs 1.86M (GPT)
- Claude's HandlePromise has no such stutter branch

EXPERIMENT 3: WHY IT EXPLODES

GPT's HandlePromise: two disjuncts

```
HandlePromise(p, m) ==  
  /\ m \in msgs  
  /\ m.kind = "Promise"  
  /\ p \in Proposers  
  /\ m.to = p  
  /\ ( \* process a fresh promise  
      /\ m.pid = proposalId[p]  
      /\ ~(m.from \in promisesRcvd[p])  
      /\ LET ... IN \* update state;  
          \* emit Accept msgs  
  /\ \* ignore duplicates or old promises  
  /\ UNCHANGED vars  
  )
```

- Disjunct (2) has no guard — every Promise can be processed OR ignored
- TLC must explore both → branching doubles on every message
- Same config, Small: 1,217 states (Claude) vs 1.86M (GPT)
- Claude's HandlePromise has no such stutter branch

That branch faithfully models message loss → 100% TV. Fidelity has a price: more accurate ≠ more efficient.

LIMITATIONS

- **Narrow protocol scope:** single-decree Essential Paxos, not full Paxos or industrial systems

LIMITATIONS

- **Narrow protocol scope:** single-decree Essential Paxos, not full Paxos or industrial systems
- **Bounded verification:** TLC proves invariants only for the checked configurations, not all configurations

LIMITATIONS

- **Narrow protocol scope:** single-decree Essential Paxos, not full Paxos or industrial systems
- **Bounded verification:** TLC proves invariants only for the checked configurations, not all configurations
- **Partial behavioral coverage:** TV uses 4 deterministic traces, so it cannot cover all executions

LIMITATIONS

- **Narrow protocol scope:** single-decree Essential Paxos, not full Paxos or industrial systems
- **Bounded verification:** TLC proves invariants only for the checked configurations, not all configurations
- **Partial behavioral coverage:** TV uses 4 deterministic traces, so it cannot cover all executions
- **Limited generality:** one protocol, two models, no repeated trials

CONCLUSION

Does feedback-driven prompt refinement improve the quality of LLM-generated TLA+ specifications, and are the resulting prompts transferable across Claude and GPT?

CONCLUSION

Does feedback-driven prompt refinement improve the quality of LLM-generated TLA+ specifications, and are the resulting prompts transferable across Claude and GPT?

- **Yes:** feedback improved both models from failing to passing; both models failed single-pass

CONCLUSION

Does feedback-driven prompt refinement improve the quality of LLM-generated TLA+ specifications, and are the resulting prompts transferable across Claude and GPT?

- **Yes:** feedback improved both models from failing to passing; both models failed single-pass
- **No:** refined prompts are brittle and model-specific, they don't transfer

CONCLUSION

Does feedback-driven prompt refinement improve the quality of LLM-generated TLA+ specifications, and are the resulting prompts transferable across Claude and GPT?

- **Yes:** feedback improved both models from failing to passing; both models failed single-pass
- **No:** refined prompts are brittle and model-specific, they don't transfer
- **Key bottleneck:** semantic conformance, not just compilation

CONCLUSION

Does feedback-driven prompt refinement improve the quality of LLM-generated TLA+ specifications, and are the resulting prompts transferable across Claude and GPT?

- **Yes:** feedback improved both models from failing to passing; both models failed single-pass
- **No:** refined prompts are brittle and model-specific, they don't transfer
- **Key bottleneck:** semantic conformance, not just compilation
- **Next step:** evaluate on richer protocols (Raft, CURP) and repeat runs to test generality

Thank you!

Ana Mako

Vuk Jurišić

Giorgos Markozanis